DHT의 탄생: 구조화된 분산의 해법

1. 들어가며

1편에서 살펴본 1세대 P2P 시스템들은 **"구조 없는 분산"**의 한계를 드러냈습니다:

- **Napster**: 중앙 서버가 단일 장애점
- **Gnutella**: O(n) flooding으로 확장성 한계
- **FastTrack**: 슈퍼노드로 개선했지만 근본적 해결 아님

2001년, MIT와 NYU의 연구자들이 이 문제에 대한 해답을 제시했습니다. **DHT(Distributed Hash Table)**는 "검색을 O(log n)으로, 그리고 데이터 위치를 예측 가능하게" 만드는 구조화된 접근법입니다.

이 글에서는 DHT의 핵심 개념과 두 가지 대표적인 프로토콜인 **Chord**와 **Kademlia**를 원본 논문을 통해 살펴봅니다.

---

2. 문제 정의: 1세대 P2P의 한계

2.1 Chord 논문의 문제 인식

2001년 SIGCOMM에서 발표된 Chord 논문은 문제를 다음과 같이 정의했습니다:

> "A fundamental problem that confronts peer-to-peer applications is to efficiently locate the node that stores a particular data item."
> — Stoica et al., "Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications", SIGCOMM 2001

논문은 기존 시스템들의 한계를 명확히 지적했습니다:

> "Chord can help avoid single points of failure or control that systems like Napster possess, and the lack of scalability that systems like Gnutella display because of their widespread use of broadcasts."
> — Stoica et al., 2001

2.2 설계 목표

Chord 논문은 5가지 설계 목표를 제시했습니다 (논문 Section 3에서 직접 인용):

> "**Load balance**: Chord acts as a distributed hash function, spreading keys evenly over the nodes; this provides a degree of natural load balance."
>
> "**Decentralization**: Chord is fully distributed: no node is more important than any other. This improves robustness and makes Chord appropriate for loosely-organized peer-to-peer applications."
>
> "**Scalability**: The cost of a Chord lookup grows as the log of the number of nodes, so even very large systems are feasible. No parameter tuning is required to achieve this scaling."
>
> "**Availability**: Chord automatically adjusts its internal tables to reflect newly joined nodes as well as node failures, ensuring that, barring major failures in the underlying network, the node responsible for a key can always be found."

> "**Flexible naming**: Chord places no constraints on the structure of the keys it looks up: the Chord key-space is flat."
> — Stoica et al., 2001, Section 3

---

3. Consistent Hashing: DHT의 기반

3.1 기원

Consistent Hashing은 1997년 David Karger 등이 웹 캐싱 문제를 해결하기 위해 제안했습니다:

> "We describe a family of caching protocols for distributed networks that can be used to decrease or eliminate the occurrence of hot spots in the network."
> — Karger et al., "Consistent Hashing and Random Trees", STOC 1997

Chord 논문에서 Consistent Hashing의 핵심 속성을 설명합니다:

> "Chord uses a variant of consistent hashing to assign keys to Chord nodes. Consistent hashing tends to balance load, since each node receives roughly the same number of keys, and involves relatively little movement of keys when nodes join and leave the system."
> — Stoica et al., 2001

3.2 핵심 아이디어

Chord 논문 Section 4.2에서 Consistent Hashing을 설명합니다:

> "The consistent hash function assigns each node and key an m-bit identifier using a base hash function such as SHA-1. A node's identifier is chosen by hashing the node's IP address, while a key identifier is produced by hashing the key."
> — Stoica et al., 2001

> "Consistent hashing assigns keys to nodes as follows. Identifiers are ordered in an identifier circle modulo 2^m. Key k is assigned to the first node whose identifier is equal to or follows (the identifier of) k in the identifier space. This node is called the **successor node** of key k, denoted by successor(k)."
> — Stoica et al., 2001

```
          0
         /|\
        / | \
       /  |  \
     7    |    1
      \   |   /
       \  |  /
    6---[Ring]---2
       /  |  \
      /   |   \
     5    |    3
       \  |  /
        \ | /
         \|/
          4
```

```
예시: m=3 (8개 식별자), 노드 0, 1, 3이 존재
- Key 1 → successor(1) = Node 1
- Key 2 → successor(2) = Node 3
- Key 6 → successor(6) = Node 0
```

3.3 부하 분산 보장

논문의 Theorem 1에서 부하 분산을 증명합니다:

> "**THEOREM 1.** For any set of N nodes and K keys, with high probability:
>
> 1. Each node is responsible for at most (1 + ε)K/N keys
> 2. When an (N + 1)st node joins or leaves the network, responsibility for O(K/N) keys changes hands (and only to or from the joining or leaving node)."
>
> — Stoica et al., 2001

이것이 Consistent Hashing의 핵심입니다: 노드가 추가/제거되어도 **O(K/N)개의 키만 재배치**됩니다.

---

4. Chord: 로그 시간 검색의 실현

4.1 기본 아이디어

Consistent Hashing만으로는 모든 노드가 서로를 알아야 합니다. Chord는 이를 개선합니다:

> "Previous work on consistent hashing assumed that nodes were aware of most other nodes in the system, making it impractical to scale to large number of nodes. In contrast, each Chord node needs 'routing' information about only a few other nodes."
> — Stoica et al., 2001

핵심 성능 보장:

> "In the steady state, in an N-node system, each node maintains information only about O(log N) other nodes, and resolves all lookups via O(log N) messages to other nodes."
> — Stoica et al., 2001

4.2 Finger Table

Chord의 핵심 자료구조는 **Finger Table**입니다. 논문에서:

> "As before, let m be the number of bits in the key/node identifiers. Each node, n, maintains a routing table with (at most) m entries, called the **finger table**. The i-th entry in the table at node n contains the identity of the first node, s, that succeeds n by at least 2^(i-1) on the identifier circle, i.e., s = successor(n + 2^(i-1)), where 1 ≤ i ≤ m."
> — Stoica et al., 2001

이것이 의미하는 바:

Finger	거리	커버하는 범위

Finger	거리	커버하는 범위
finger[1]	2^0 = 1	바로 다음 노드
finger[2]	2^1 = 2	1/2 지점
finger[3]	2^2 = 4	1/4 지점
...	...	...
finger[m]	2^(m-1)	반대편

> "This scheme has two important characteristics. First, each node stores information about only a small number of other nodes, and knows more about nodes closely following it on the identifier circle than about nodes farther away."
> — Stoica et al., 2001

4.3 검색 알고리즘

논문의 Figure 4에서 검색 알고리즘을 제시합니다:

```
// ask node n to find id's successor
n.find_successor(id)
    n' = find_predecessor(id);
    return n'.successor;

// ask node n to find id's predecessor
n.find_predecessor(id)
    n' = n;
    while (id ∉ (n', n'.successor])
        n' = n'.closest_preceding_finger(id);
    return n';

// return closest finger preceding id
n.closest_preceding_finger(id)
    for i = m downto 1
        if (finger[i].node ∈ (n, id))
            return finger[i].node;
    return n;
```

출처: Stoica et al., 2001, Figure 4

4.4 로그 시간 증명

논문의 Theorem 2에서 성능을 증명합니다:

> "**THEOREM 2.** With high probability (or under standard hardness assumptions), the number of nodes that must be contacted to find a successor in an N-node network is O(log N)."
> — Stoica et al., 2001

증명의 핵심 아이디어:

> "Suppose that node n wishes to resolve a query for the successor of k. ... If node n contacts a node n' such that the i-th finger interval of node n, then since this interval is not empty, node n will finger some node f in this interval. The distance (number of identifiers) between f and k is at least 2^(i-1). But f and k are both in n's i-th finger interval, which means the distance between them is at most 2^i. This means f is closer to k than to n, or equivalently, that **the distance from f to k is at most half the distance from n to k**."
> — Stoica et al., 2001

매 홉마다 거리가 절반으로 줄어드므로 → **O(log N)** 홉

4.5 실험 결과

논문 Section 6에서 시뮬레이션 결과를 제시합니다:

> "Figure 10(a) shows that the path length is about (1/2) log N. The reason for the 1/2 is as follows. ... In general, the number of fingers we need to follow will be the number of ones in the binary representation of the distance from node to query. Since the distance is random, we expect half the log N bits to be ones."
> — Stoica et al., 2001

실제 측정에서:

> "The median latency ranges from 180 to 285 ms, depending on number of nodes. For the case of 180 nodes, a typical lookup involves five two-way message exchanges: four for the Chord lookup, and a final message to the successor node."
> — Stoica et al., 2001

---

5. Kademlia: XOR 메트릭의 혁신

5.1 배경

2002년, Petar Maymounkov와 David Mazières가 **Kademlia**를 제안했습니다:

> "We describe a peer-to-peer system which has provable consistency and performance in a fault-prone environment. Our system routes queries and locates nodes using a novel **XOR-based metric topology** that simplifies the algorithm and facilitates our proof."
> — Maymounkov & Mazières, "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric", IPTPS 2002

5.2 XOR 메트릭

Kademlia의 핵심 혁신은 XOR 기반 거리 측정입니다:

> "Given two 160-bit identifiers, x and y, Kademlia defines the distance between them as their bitwise exclusive or (XOR) interpreted as an integer, d(x, y) = x ⊕ y."
> — Maymounkov & Mazières, 2002

XOR가 유효한 메트릭임을 증명합니다:

> "We first note that XOR is a valid, albeit non-Euclidean, metric. It is obvious that d(x,x) = 0, d(x,y) > 0 if x ≠ y, and ∀x,y : d(x,y) = d(y,x). XOR also offers the triangle property: d(x,y) + d(y,z) ≥ d(x,z)."
> — Maymounkov & Mazières, 2002

5.3 XOR의 장점: 대칭성

Chord와의 핵심 차이점:

> "XOR is symmetric, allowing Kademlia participants to receive lookup queries from precisely the same distribution of nodes contained in their routing tables. Without this property, systems such as Chord do not learn useful routing information from queries they receive."
> — Maymounkov & Mazières, 2002

Chord의 한계를 지적합니다:

> "Worse yet, because of the asymmetry of Chord's metric, Chord routing tables are rigid. Each entry in a Chord node's finger table must store the precise node proceeding an interval in the ID space; any node actually in the interval will be greater than some keys in the interval, and thus very far from the key. **Kademlia, in contrast, can send a query to any node within an interval**, allowing it to select routes based on latency or even send parallel asynchronous queries."
> — Maymounkov & Mazières, 2002

5.4 k-bucket: 라우팅 테이블 구조

> "For each 0 ≤ i < 160, every node keeps a list of ⟨IP address, UDP port, Node ID⟩ triples for nodes of distance between 2^i and 2^(i+1) from itself. We call these lists **k-buckets**. Each k-bucket is kept sorted by time last seen—least-recently seen node at the head, most-recently seen at the tail."
> — Maymounkov & Mazières, 2002

k의 의미:

> "k is a system-wide replication parameter. k is chosen such that any given k nodes are very unlikely to fail within an hour of each other (for example k = 20)."
> — Maymounkov & Mazières, 2002

5.5 노드 업타임 기반 최적화

Kademlia의 독특한 특징은 **노드 업타임 분석**에 기반한 최적화입니다:

> "k-buckets effectively implement a least-recently seen eviction policy, except that live nodes are never removed from the list. This preference for old contacts is driven by our analysis of Gnutella trace data collected by Saroiu et al."
> — Maymounkov & Mazières, 2002

논문의 Figure 1에서:

> "The longer a node has been up, the more likely it is to remain up another hour. By keeping the oldest live contacts around, k-buckets maximize the probability that the nodes they contain will remain online."
> — Maymounkov & Mazières, 2002

5.6 프로토콜: 4가지 RPC

> "The Kademlia protocol consists of four RPCs: **PING**, **STORE**, **FIND_NODE**, and **FIND_VALUE**."
> — Maymounkov & Mazières, 2002

각 RPC의 역할:

RPC	설명 (논문 발췌)
PING	"probes a node to see if it is online"
STORE	"instructs a node to store a ⟨key, value⟩ pair for later retrieval"
FIND_NODE	"The recipient of the RPC returns ⟨IP address, UDP port, Node ID⟩ triples for the k nodes it knows about closest to the target ID"
FIND_VALUE	"behaves like FIND_NODE... with one exception. If the RPC recipient has received a STORE RPC for the key, it just returns the stored value"

출처: Maymounkov & Mazières, 2002

5.7 병렬 비동기 검색

Kademlia의 또 다른 혁신:

> "The lookup initiator starts by picking α nodes from its closest non-empty k-bucket... The initiator then sends **parallel, asynchronous FIND_NODE RPCs** to the α nodes it has chosen. α is a system-wide concurrency parameter, such as 3."
> — Maymounkov & Mazières, 2002

α의 역할:

> "When α = 1 the lookup algorithm resembles Chord's in terms of message cost and the latency of detecting failed nodes. However, Kademlia can route for lower latency because it has the flexibility of choosing any one of k nodes to forward a request to."
> — Maymounkov & Mazières, 2002

5.8 DoS 공격 저항성

k-bucket의 보안 이점:

> "A second benefit of k-buckets is that they provide resistance to certain DoS attacks. One cannot flush nodes' routing state by flooding the system with new nodes. Kademlia nodes will only insert the new nodes in the k-buckets when old nodes leave the system."
> — Maymounkov & Mazières, 2002

---

6. Chord vs Kademlia: 비교 분석

6.1 메트릭 비교

속성	Chord	Kademlia

속성	Chord	Kademlia
거리 함수	시계방향 거리 (mod 2^m)	XOR (x ⊕ y)
대칭성	비대칭	대칭
라우팅 유연성	고정 (finger table)	유연 (k-bucket 내 선택)

6.2 Kademlia의 장점 요약

Kademlia 논문의 결론에서:

> "With its novel XOR-based metric topology, Kademlia is the first peer-to-peer system to combine provable consistency and performance, latency-minimizing routing, and a symmetric, unidirectional topology. Kademlia furthermore introduces a concurrency parameter, α, that lets people trade a constant factor in bandwidth for asynchronous lowest-latency hop selection and delay-free fault recovery. Finally, Kademlia is the first peer-to-peer system to exploit the fact that **node failures are inversely related to uptime**."
> — Maymounkov & Mazières, 2002

6.3 실제 적용

Wikipedia에 따르면 Kademlia 기반 시스템들:

> "Mainline DHT: a DHT for BitTorrent based on an implementation of the Kademlia algorithm, for trackerless torrents."
> — Wikipedia, Kademlia

다른 적용 사례들:

- **BitTorrent Mainline DHT**: 트래커 없는 토렌트
- **IPFS**: libp2p의 DHT 구현
- **Ethereum**: 노드 디스커버리

---

7. DHT의 한계와 트레이드오프

7.1 정확한 키 검색만 지원

DHT는 정확한 키 매칭만 지원합니다:

> "Chord provides support for just one operation: given a key, it maps the key onto a node."
> — Stoica et al., 2001

이것이 의미하는 바:

- ☑ "파일 해시 abc123의 위치는?" → 가능
- ❌ "비틀즈 노래 찾아줘" (키워드 검색) → 불가능

7.2 Churn 문제

노드의 빈번한 가입/탈퇴는 DHT에 부담입니다. Chord 논문에서:

> "We assume that any two nodes trying to communicate will eventually succeed."
> — Stoica et al., 2001

실제 P2P 네트워크에서는 이 가정이 항상 성립하지 않습니다.

7.3 보안 취약점

Sybil Attack: 악의적 노드가 다수의 가짜 ID를 생성하여 네트워크를 장악할 수 있습니다.

Eclipse Attack: 특정 노드의 라우팅 테이블을 악의적 노드들로 채워 네트워크에서 고립시킬 수 있습니다.

---

8. 1세대 P2P에서 DHT로: 복잡도 비교

시스템	검색 복잡도	노드당 상태	검색 보장
Napster	O(1)	O(1)	있음 (중앙 서버)
Gnutella	O(N)	O(이웃 수)	없음
FastTrack	O(√N)	O(√N)	없음
Chord	O(log N)	O(log N)	있음
Kademlia	O(log N)	O(log N)	있음

핵심 진보:

- **O(N) → O(log N)**: 백만 노드에서 20홉 이내로 검색 가능
- **검색 보장**: 데이터가 존재하면 반드시 찾을 수 있음
- **분산 유지**: 중앙 서버 없이 달성

---

9. 마치며

DHT는 P2P 네트워킹의 **패러다임 전환**이었습니다.

Chord 논문의 결론:

> "Many distributed peer-to-peer applications need to determine the node that stores a data item. The Chord protocol solves this challenging problem in decentralized manner. It offers a powerful primitive: given a key, it determines the node responsible for storing the key's value, and does so efficiently."
> — Stoica et al., 2001

Kademlia 논문의 결론:

> "With its novel XOR-based metric topology, Kademlia is the first peer-to-peer system to combine provable consistency and performance, latency-minimizing routing, and a symmetric, unidirectional topology."
> — Maymounkov & Mazières, 2002

핵심 통찰:

> **"구조화된 분산은 구조 없는 분산보다 강력하다."**

DHT는 오늘날 BitTorrent, IPFS, 블록체인 네트워크의 기반이 되었습니다. 다음 편에서는 DHT를 실제로 활용한 **BitTorrent**의 혁신을 살펴봅니다.

참고 자료

논문 (직접 인용)

1. **Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. (2001).** "Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications." *ACM SIGCOMM 2001*.
    - 원문 PDF: https://pdos.csail.mit.edu/papers/chord:sigcomm01/chord_sigcomm.pdf
    - ACM SIGCOMM Test of Time Award 2011 수상
2. **Maymounkov, P., & Mazières, D. (2002).** "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric." *IPTPS 2002*.
    - 원문 PDF: https://www.scs.stanford.edu/~dm/home/papers/kpos.pdf
3. **Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997).** "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web." *ACM STOC 1997*.

백과사전 및 참고 자료

4. **Wikipedia**: Chord (peer-to-peer) - https://en.wikipedia.org/wiki/Chord_(peer-to-peer)
5. **Wikipedia**: Kademlia - https://en.wikipedia.org/wiki/Kademlia
6. **Wikipedia**: Distributed hash table - https://en.wikipedia.org/wiki/Distributed_hash_table